Status	Finished
Started	Tuesday, 27 October 2026, 12:18 PM
Completed	Tuesday, 27 October 2026, 12:24 PM
Duration	5 mins 50 secs
Grade	**12.00** out of 15.00 (**80**%)

Question **1**

Complete

Mark 1.00 out of 1.00

Which of the following is a key reason for the importance of sorting algorithms?

a. Sorting decreases the efficiency of selection operations
b. Sorting helps in finding duplicates quickly
c. Sorting makes it harder to search for items
d. Sorting is rarely used in programming

The correct answer is: Sorting helps in finding duplicates quickly

Question **2**

Complete

Mark 1.00 out of 1.00

The process of placing or rearranging a collection of elements into a particular order is known as

a. Sorting
b. Rearranging
c. Searching
d. Merging

The correct answer is: Sorting

Question **3**

Complete

Mark 1.00 out of 1.00

Which sorting algorithm would be preferred for its divide-and-conquer approach?

- a. Merge Sort
- b. Binary Search
- c. Bubble Sort
- d. Linear Search

The correct answer is: Merge Sort

Question **4**

Complete

Mark 0.00 out of 1.00

Very slow way of sorting is_____

- a. Bubble sort
- b. Heap sort
- c. Insertion sort
- d. Quick sort

The correct answer is:
Insertion sort

Question **5**

Complete

Mark 1.00 out of 1.00

Which sorting algorithm is described as making multiple passes through a list, comparing elements, and swapping adjacent items that are out of order?

- a. Insertion Sort
- b. Merge Sort
- c. Bubble Sort
- d. Quick Sort

The correct answer is: Bubble Sort

Question **6**

Complete

Mark 1.00 out of 1.00

How does Merge Sort achieve its efficiency?

a. By comparing elements sequentially
b. By breaking the input into smaller parts and merging them
c. By using the bubble-up method
d. By sorting data in a single pass

The correct answer is: By breaking the input into smaller parts and merging them

Question **7**

Complete

Mark 1.00 out of 1.00

Which sorting algorithm involves comparing elements and swapping adjacent items that are out of order?

a. Bubble Sort
b. Binary Search
c. Merge Sort
d. Linear Search

The correct answer is: Bubble Sort

Question **8**

Complete

Mark 1.00 out of 1.00

_____is putting an element in the appropriate place in a sorted list yields a larger sorted order list.

a. Distribution
b. Insertion
c. Selection
d. Extraction

The correct answer is:
Insertion

Question **9**

Complete

Mark 1.00 out of 1.00

In the context of sorting, what does the divide-and-conquer approach involve?

a. Sorting data in a single pass
b. Dividing the input into parts, solving each part, and combining the solutions
c. Sorting data sequentially
d. Rearranging data without sorting

The correct answer is: Dividing the input into parts, solving each part, and combining the solutions

Question **10**

Complete

Mark 1.00 out of 1.00

What is one advantage of sorting a list before performing a search operation?

a. It has no effect on the search operation
b. It increases the number of comparisons needed
c. It allows for faster searching
d. It makes the search operation slower

The correct answer is: It allows for faster searching

Question **11**

Complete

Mark 1.00 out of 1.00

What is one of the key advantages of using the built-in sorted() function in Python?

a. It is less efficient than custom sorting algorithms
b. It only works with integer arrays
c. It sorts data out of the box efficiently
d. It requires external libraries

The correct answer is: It sorts data out of the box efficiently

Question **12**

Complete

Mark 1.00 out of 1.00

Algorithm design technique used in merge sort algorithm is

a. Greedy method
b. Backtracking
c. Divide and conquer
d. Dynamic programming

The correct answer is:
Divide and conquer

Question **13**

Complete

Mark 0.00 out of 1.00

Two-way merge sort algorithm is used to sort the following elements in ascending order.
200,470,150,80,90,40,400,300,120,70
What is the order of these elements after second pass of the merge sort algorithm?

a. 40,70,80,90,120,150,200,300,400,470
b. 80,150,200,470,40,90,300,400,70,120
c. 40,80,90,150,200,300,400,470,70,120
d. 200,470,80,150,40,90,300,400,70,120

The correct answer is:
80,150,200,470,40,90,300,400,70,120

Question **14**

Complete

Mark 1.00 out of 1.00

What does the Bubble Sort algorithm primarily focus on during each pass?

a. Sorting the entire list in one pass
b. Dividing the list into halves
c. Bubbling up the largest element to its correct position
d. Bubbling up the smallest element

The correct answer is: Bubbling up the largest element to its correct position

Question **15**

Complete

Mark 0.00 out of 1.00

In Merge Sort, what happens after dividing the input into smaller parts?

a. Each part is searched for a specific element
b. Each part is sorted independently
c. The parts are ignored
d. The parts are merged without sorting

The correct answer is: Each part is sorted independently

Status	Finished
Started	Thursday, 29 October 2026, 12:43 PM
Completed	Thursday, 29 October 2026, 1:16 PM
Duration	32 mins 56 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given an listof integers, sort the array in ascending order using the *Bubble Sort* algorithm above. Once sorted, print the following three lines:

1. List is sorted in numSwaps swaps., where numSwaps is the number of swaps that took place.
2. First Element: firstElement, the *first* element in the sorted list.
3. Last Element: lastElement, the *last* element in the sorted list.

For example, given a worst-case but small array to sort: a=[6,4,1]. It took 3 swaps to sort the array. Output would be

```
Array is sorted in 3 swaps.
```

```
First Element: 1
```

```
Last Element: 6
```

Input Format

The first line contains an integer,n , the size of the list a .
The second line contains n, space-separated integers a[i].

Constraints

- 2<=n<=600
- 1<=a[i]<=$2x10^6$.

Output Format

You must print the following three lines of output:

1. List is sorted in numSwaps swaps., where numSwaps is the number of swaps that took place.
2. First Element: firstElement, the *first* element in the sorted list.
3. Last Element: lastElement, the *last* element in the sorted list.

Sample Input 0

3

1 2 3

Sample Output 0

List is sorted in 0 swaps.

First Element: 1

Last Element: 3

For example:

Input	Result
3 3 2 1	List is sorted in 3 swaps. First Element: 1 Last Element: 3
5 1 9 2 8 4	List is sorted in 4 swaps. First Element: 1 Last Element: 9

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    a = [int(x) for x in input_data[1:n+1]]

    numSwaps = 0
```

```
    # Bubble Sort implementation
    for i in range(n):
        for j in range(n - 1):
            if a[j] > a[j + 1]:
                # Swap elements
                a[j], a[j + 1] = a[j + 1], a[j]
                numSwaps += 1

    # Print results in the exact requested format
    print(f"List is sorted in {numSwaps} swaps.")
    print(f"First Element: {a[0]}")
    print(f"Last Element: {a[-1]}")

if __name__ == "__main__":
    solve()
```

	Input	Expected	Got	
✓	3 3 2 1	List is sorted in 3 swaps. First Element: 1 Last Element: 3	List is sorted in 3 swaps. First Element: 1 Last Element: 3	✓
✓	5 1 9 2 8 4	List is sorted in 4 swaps. First Element: 1 Last Element: 9	List is sorted in 4 swaps. First Element: 1 Last Element: 9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order. You read an list of numbers. You need to arrange the elements in ascending order and print the result. The sorting should be done using bubble sort.

Input Format: The first line reads the number of elements in the array. The second line reads the array elements one by one.

Output Format: The output should be a sorted list.

For example:

Input	Result
6 3 4 8 7 1 2	1 2 3 4 7 8
5 4 5 2 3 1	1 2 3 4 5

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    a = [int(x) for x in input_data[1:n+1]]

    # Bubble Sort implementation
    for i in range(n):
        for j in range(0, n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]

    # Print the sorted elements separated by a space
    print(*(a))

if __name__ == "__main__":
    solve()
```

	Input	Expected	Got	
✓	6 3 4 8 7 1 2	1 2 3 4 7 8	1 2 3 4 7 8	✓
✓	6 9 18 1 3 4 6	1 3 4 6 9 18	1 3 4 6 9 18	✓
✓	5 4 5 2 3 1	1 2 3 4 5	1 2 3 4 5	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Write a Python program to sort a list of elements using the merge sort algorithm.

For example:

Input	Result
5 6 5 4 3 8	3 4 5 6 8

Answer: (penalty regime: 0 %)

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    return result + left[i:] + right[j:]

n = int(input())
arr = list(map(int, input().split()))
print(*merge_sort(arr))
```

	Input	Expected	Got	
✓	5 6 5 4 3 8	3 4 5 6 8	3 4 5 6 8	✓
✓	9 14 46 43 27 57 41 45 21 70	14 21 27 41 43 45 46 57 70	14 21 27 41 43 45 46 57 70	✓
✓	4 86 43 23 49	23 43 49 86	23 43 49 86	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

To find the frequency of numbers in a list and display in sorted order.

Constraints:

1<=n, arr[i]<=100

Input:

1 68 79 4 90 68 1 4 5

output:

1 2

4 2

5 1

68 2

79 1

90 1

For example:

Input	Result
4 3 5 3 4 5	3 2 4 2 5 2

Answer: (penalty regime: 0 %)

```
from collections import Counter

arr = list(map(int, input().split()))
freq = Counter(arr)

for num in sorted(freq):
    print(num, freq[num])

```

	Input	Expected	Got	
✓	4 3 5 3 4 5	3 2 4 2 5 2	3 2 4 2 5 2	✓
✓	12 4 4 4 2 3 5	2 1 3 1 4 3 5 1 12 1	2 1 3 1 4 3 5 1 12 1	✓

	Input	Expected	Got	
✓	5 4 5 4 6 5 7 3	3 1 4 2 5 3 6 1 7 1	3 1 4 2 5 3 6 1 7 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

The problem is that we want to reverse a array in O(N) linear time complexity and we want the algorithm to be in-place as well!
For example: input is [1,2,3,4,5] then the output is [5,4,3,2,1]
Input
5
1 2 3 4 5
Output
5 4 3 2 1

For example:

Input	Result
5 1 2 3 4 5	5 4 3 2 1

Answer: (penalty regime: 0 %)

```
n = int(input())
arr = list(map(int, input().split()))

left, right = 0, n - 1
while left < right:
    arr[left], arr[right] = arr[right], arr[left]
    left += 1
    right -= 1

print(*arr)
```

	Input	Expected	Got	
✓	5 1 2 3 4 5	5 4 3 2 1	5 4 3 2 1	✓
✓	10 0 2 4 6 8 1 3 5 7 9	9 7 5 3 1 8 6 4 2 0	9 7 5 3 1 8 6 4 2 0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order*.

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- `1 <= nums.length <= 10^4`
- `-10^4 <= nums[i] <= 10^4`
- `nums` is sorted in **non-decreasing** order.

For example:

Test	Result
`print(sortedSquares([-4,-1,0,3,10]))`	`[0, 1, 9, 16, 100]`

Answer: (penalty regime: 0 %)

Reset answer

```
def sortedSquares(nums):
    return sorted(x * x for x in nums)

```

	Test	Expected	Got	
✓	`print(sortedSquares([-4,-1,0,3,10]))`	`[0, 1, 9, 16, 100]`	`[0, 1, 9, 16, 100]`	✓
✓	`print(sortedSquares([-7,-3,2,3,11]))`	`[4, 9, 9, 49, 121]`	`[4, 9, 9, 49, 121]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given an integer array `nums`, return *an integer array* `counts` *where* `counts[i]` *is the number of smaller elements to the right of* `nums[i]`.

Example 1:

```
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.
```

Example 2:

```
Input: nums = [-1]
Output: [0]
```

Example 3:

```
Input: nums = [-1,-1]
Output: [0,0]
```

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`

For example:

Test	Result
`print(countSmaller([5,2,6,1]))`	`[2, 1, 1, 0]`
`print(countSmaller([-1]))`	`[0]`

Answer: (penalty regime: 0 %)

Reset answer

```
def countSmaller(nums):
    return [sum(1 for y in nums[i+1:] if y < nums[i]) for i in range(len(nums))]
```

	Test	Expected	Got	
✓	`print(countSmaller([5,2,6,1]))`	`[2, 1, 1, 0]`	`[2, 1, 1, 0]`	✓
✓	`print(countSmaller([50,20,60,10]))`	`[2, 1, 1, 0]`	`[2, 1, 1, 0]`	✓

	Test	Expected	Got	
✓	`print(countSmaller([-1]))`	`[0]`	`[0]`	✓
✓	`print(countSmaller([-1, -1]))`	`[0, 0]`	`[0, 0]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given an array of integers `nums`, sort the array in **increasing** order based on the frequency of the values. If multiple values have the same frequency, sort them in **decreasing** order.

Print the *sorted array*.

Example 1:

```
Input:
6
1 1 2 2 2 3

Output:
3 1 1 2 2 2

Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.
```

Example 2:

```
Input:
5
2 3 1 3 2
Output:
1 3 3 2 2
Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.
```

Example 3:

```
Input:
9
-1 1 -6 4 5 -6 1 4 1
Output:
5 -1 4 4 -6 -6 1 1 1
```

Constraints:

- `1 <= nums.length <= 100`
- `-100 <= nums[i] <= 100`

For example:

Input	Result
6 1 1 2 2 2 3	3 1 1 2 2 2
5 2 3 1 3 2	1 3 3 2 2

Answer: (penalty regime: 0 %)

```
from collections import Counter

n = int(input())
nums = list(map(int, input().split()))
freq = Counter(nums)

nums.sort(key=lambda x: (freq[x], -x))
print(*nums)
```

	Input	Expected	Got	
✓	6 1 1 2 2 2 3	3 1 1 2 2 2	3 1 1 2 2 2	✓
✓	5 2 3 1 3 2	1 3 3 2 2	1 3 3 2 2	✓
✓	9 -1 1 -6 4 5 -6 1 4 1	5 -1 4 4 -6 -6 1 1 1	5 -1 4 4 -6 -6 1 1 1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Objective:

Develop a Python program to find the k-th maximum value in a given list of integers. The program should handle various edge cases, including lists with duplicate values, empty lists, and invalid values of k. The k-th maximum value refers to the k-th largest distinct element in the list.

Background:

Finding the k-th maximum value in a list is a common problem in computer science, often encountered in fields like data analysis, competitive programming, and software development. This problem requires an understanding of sorting algorithms, data structures, and efficient problem-solving techniques. By solving this problem, one gains insights into how to handle large datasets and optimize performance in practical applications.

Problem Description:

Given a list of integers, the task is to determine the k-th maximum value in the list. The program should meet the following requirements:

1. **Input:**
    - A list of integers, which can contain both positive and negative values.
    - An integer k, representing the position of the maximum value to find.
2. **Output:**
    - The k-th maximum value in the list.
    - If k is greater than the number of distinct elements in the list or if the list is empty, the program should return an appropriate message indicating the error.

Constraints:

- The list may contain duplicate values.
- The value of k should be a positive integer.
- The list may contain up to 10^6 elements, and each element can be as large as 10^9 in magnitude.

Examples:

Consider the following examples for better understanding:

1. **Example 1:**
    - **Input:** list = [3, 1, 5, 4, 2], k = 2
    - **Output:** 4
    - **Explanation:** The distinct elements in the list are [1, 2, 3, 4, 5]. The 2nd maximum value is 4.
2. **Example 2:**
    - **Input:** list = [7, 7, 7, 7, 7], k = 1
    - **Output:** 7
    - **Explanation:** The distinct elements in the list are [7]. The 1st maximum value is 7.
3. **Example 3:**
    - **Input:** list = [2, 1, 2, 1, 2], k = 3
    - **Output:** -1
    - **Explanation:** The distinct elements in the list are [1, 2]. There is no 3rd maximum value.

For example:

Input	Result
5 3 1 5 4 2 2	4
6 7 7 7 7 7 7 1	7
10 2 1 2 1 2 1 2 1 2 1 3	-1

Answer: (penalty regime: 0 %)

```
n = int(input())
nums = list(map(int, input().split()))
k = int(input())

nums = sorted(set(nums), reverse=True)

if k <= 0 or k > len(nums):
    print(-1)
else:
    print(nums[k - 1])

```

	Input	Expected	Got	
✓	5 3 1 5 4 2 2	4	4	✓
✓	6 7 7 7 7 7 7 1	7	7	✓
✓	10 2 1 2 1 2 1 2 1 2 1 3	-1	-1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given an array of integers `arr`, replace each element with its rank.

The rank represents how large the element is. The rank has the following rules:

- Rank is an integer starting from 1.
- The larger the element, the larger the rank. If two elements are equal, their rank must be the same.
- Rank should be as small as possible.

Example 1:

```
Input: arr = [40,10,20,30]
Output: [4,1,2,3]
Explanation: 40 is the largest element. 10 is the smallest. 20 is the second smallest. 30 is the third smallest.
```

Example 2:

```
Input: arr = [100,100,100]
Output: [1,1,1]
Explanation: Same elements share the same rank.
```

Example 3:

```
Input: arr = [37,12,28,9,100,56,80,5,12]
Output: [5,3,4,2,8,6,7,1,3]
```

Constraints:

- $0 <= arr.length <= 10^5$
- $-10^9 <= arr[i] <= 10^9$

For example:

Test	Result
`print(arrayRankTransform([40,10,20,30]))`	`[4, 1, 2, 3]`

Answer: (penalty regime: 0 %)

Reset answer

```
def arrayRankTransform(arr):
    ranks = {v: i + 1 for i, v in enumerate(sorted(set(arr)))}
    return [ranks[x] for x in arr]


```

	Test	Expected	Got	
✓	`print(arrayRankTransform([40,10,20,30]))`	`[4, 1, 2, 3]`	`[4, 1, 2, 3]`	✓
✓	`print(arrayRankTransform([100,100,100]))`	`[1, 1, 1]`	`[1, 1, 1]`	✓
✓	`print(arrayRankTransform([37,12,28,9,100,56,80,5,12]))`	`[5, 3, 4, 2, 8, 6, 7, 1, 3]`	`[5, 3, 4, 2, 8, 6, 7, 1, 3]`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.